

Simulacion de Evaluacion Practica - C#

Indicaciones generales

- Resolver cada ejercicio en un proyecto de consola de C#.
- Usar nombres de variables claros y mantener el código ordenado.
- Validar entradas numéricas con TryParse para evitar que el programa falle.
- Separar los datos en clases cuando el ejercicio lo solicite.
- No colocar Console.WriteLine dentro de las clases de modelo; la interacción con el usuario debe estar en Program.cs.
- Los ejercicios difíciles deben guardar o leer información desde archivos.

Nivel Basico

Ejercicio 1: Sistema de Control de Combustible

Objetivo: Desarrollar un programa que simule el consumo de combustible de una motocicleta.

Requisitos:

- La motocicleta inicia con 20 litros de combustible.
- El tanque tiene capacidad máxima de 25 litros.
- Mostrar un menú repetitivo con las opciones: conducir, recargar combustible, ver estado y salir.
- La opción conducir debe pedir kilómetros a recorrer.
- Cada kilómetro consume 0.1 litros.
- No permitir conducir si no hay combustible suficiente.
- La opción recargar debe pedir litros y no permitir superar la capacidad máxima.
- La opción ver estado debe mostrar litros actuales y porcentaje del tanque.
- Debe usar while, switch, if y TryParse.

Criterio de evaluación: el programa debe compilar, validar entradas, resolver el problema solicitado y mostrar mensajes claros al usuario.

Ejercicio 2: Validador de Correo Institucional

Objetivo: Crear un programa que solicite un correo institucional y no permita avanzar hasta que sea válido.

Requisitos:

- El correo debe tener al menos 10 caracteres.
- Debe contener el símbolo @.
- Debe terminar en .edu.sv.
- No debe contener espacios.
- Cuando falle una regla, mostrar exactamente la regla incumplida.
- El programa debe seguir solicitando el correo hasta que sea válido.
- Debe usar Length, Contains, EndsWith, Trim y un bucle.



Criterio de evaluacion: el programa debe compilar, validar entradas, resolver el problema solicitado y mostrar mensajes claros al usuario.

Nivel Intermedio

Ejercicio 3: Sistema de Inventario de Tienda

Objetivo: Crear un CRUD en memoria para administrar productos de una tienda.

Requisitos:

- Crear una clase Producto conCodigo, Nombre, Precio y Stock.
- Crear un menu con: agregar, mostrar, buscar, actualizar stock, eliminar y salir.
- El codigo del producto no puede repetirse.
- El precio debe ser mayor que cero.
- El stock no puede ser negativo.
- Usar List<Producto>, Find, Any o Remove.
- La clase Producto no debe tener Console.WriteLine.

Criterio de evaluacion: el programa debe compilar, validar entradas, resolver el problema solicitado y mostrar mensajes claros al usuario.

Ejercicio 4: Sistema de Citas Medicas

Objetivo: Simular una cola de pacientes que esperan atencion medica.

Requisitos:

- using System.Collections.Generic;
- Crear una cola:
`Queue<MiClase> cola = new Queue<MiClase>();`
- Agregar a la cola:
`cola.Enqueue(objeto);`
- Sacar el primero de la cola:
`MiClase siguiente = cola.Dequeue();`
- Ver el primero sin sacarlo:
`MiClase siguiente = cola.Peek();`
- Saber cuántos hay en espera:
`cola.Count`
- Validar si hay elementos antes de usar Dequeue() o Peek():
`if (cola.Count > 0)`
`{`
`MiClase siguiente = cola.Dequeue();`
`}`



- Crear una clase Paciente con NumeroCita, Nombre y Especialidad.
- Usar Queue<Paciente> para administrar la cola.
- La opción registrar paciente debe asignar automáticamente el número de cita.
- La opción atender paciente debe sacar al primero de la cola usando Dequeue.
- La opción ver siguiente debe mostrar el primero sin sacarlo usando Peek.
- Mostrar la cantidad de pacientes en espera.
- Validar cuando la cola esté vacía.

Criterio de evaluación: el programa debe compilar, validar entradas, resolver el problema solicitado y mostrar mensajes claros al usuario.

Nivel Difícil

Ejercicio 5: Sistema de Registro de Ventas

Objetivo: Leer ventas desde un archivo de texto y generar un reporte.

Requisitos:

- Crear un archivo ventas.txt con formato Vendedor|Monto.
- Crear una clase Venta con Vendedor y Monto.
- Leer el archivo usando StreamReader.
- Separar los datos usando Split("|").
- Saltar líneas incorrectas sin cerrar el programa.
- Calcular total vendido, venta más alta y promedio.
- Agrupar ventas por vendedor usando LINQ.
- Guardar los resultados en reporte.txt usando StreamWriter.

Criterio de evaluación: el programa debe compilar, validar entradas, resolver el problema solicitado y mostrar mensajes claros al usuario.

Ejercicio 6: Gestor de Productos JSON

Objetivo: Crear un gestor de productos que conserve los datos al cerrar el programa.

Requisitos:

- Crear una clase Producto con Código, Nombre, Precio y Existencia.
- Al iniciar, cargar productos.json si existe.
- Si no existe, iniciar con una lista vacía.
- Permitir agregar, mostrar, buscar, eliminar, guardar y salir.
- Al salir, guardar automáticamente en productos.json.
- using System.Text.Json;
- Para convertir una lista de objetos a JSON:
JsonSerializer.Serialize(lista);
- Para convertir con formato ordenado:



- JsonSerializer.Serialize(lista, new JsonSerializerOptions
{
 WriteIndented = true
});
- Para convertir un JSON a una lista de objetos:
- JsonSerializer.Deserialize<List<MiClase>>(contenido);

Criterio de evaluacion: el programa debe compilar, validar entradas, resolver el problema solicitado y mostrar mensajes claros al usuario.

Nivel Integrador

Ejercicio 7: Sistema de Biblioteca

Objetivo: Crear un sistema completo para registrar libros, usuarios y prestamos con persistencia JSON.

Requisitos:

- Crear las clases Libro, Usuario y Prestamo.
- Registrar libros con codigo y titulo.
- Registrar usuarios con id y nombre.
- Prestar un libro solamente si existe y no esta prestado.
- Devolver un libro prestado.
- Mostrar libros disponibles.
- Mostrar prestamos activos.
- Guardar toda la informacion en biblioteca.json.
- Usar List<T>, LINQ, clases, validaciones, archivos y JSON.

Criterio de evaluacion: el programa debe compilar, validar entradas, resolver el problema solicitado y mostrar mensajes claros al usuario.

Rubrica sugerida

Criterio	Puntos	Descripcion
Funcionalidad	40	El programa cumple todas las operaciones solicitadas.
Validaciones	20	Evita errores por datos invalidos y usa TryParse cuando corresponde.
Estructura	20	Usa clases, listas, colas o archivos segun el nivel.
Comentarios y claridad	10	Codigo ordenado, nombres claros y comentarios utiles.
Persistencia	10	En ejercicios dificiles, los datos se guardan y se recuperan correctamente.

Archivos incluidos en el ZIP: cada carpeta contiene un proyecto de consola con Program.cs comentado y su archivo .csproj para ejecutarlo con dotnet run.